Algorithme

liam Vuilleumier

2026

1 Introduction

Short resume of the course algorithm from EPFL BA4, course given by Ola Svenson

2 Lecture 1 : Introduction

2.1 Qu'est-ce qu'un algorithme ?

Un algorithme est une procédure de calcul bien définie qui prend une ou plusieurs valeurs en **entrée** (input) et produit une ou plusieurs valeurs en **sortie** (output). C'est une séquence d'étapes de calcul transformant l'entrée en sortie.

On distingue l'algorithme de la **structure de données**, qui est une méthode pour stocker et organiser l'information afin de faciliter son accès et sa modification.

2.2 Exemple introductif : La Somme Arithmétique

Le problème est de calculer la somme des entiers de 1 à n : $\sum_{i=1}^{n} i$.

- **Approche Naïve :** Une boucle qui additionne les nombres un par un.
- **Approche Intelligente (Gauss) :** Utilisation de la formule fermée $\frac{n(n+1)}{2}$.

Analyse comparative : L'algorithme naïf effectue n additions, sa complexité dépend linéairement de la taille de l'entrée. L'approche de Gauss effectue un nombre constant d'opérations (une multiplication, une addition, une division), quelle que soit la taille de n.

2.3 Pourquoi l'efficacité compte ?

Un ordinateur lent exécutant un bon algorithme (ex: $O(1)$ ou $O(n \log n)$) finira par battre un ordinateur très puissant exécutant un mauvais algorithme (ex: $O(n^2)$) lorsque la taille de l'entrée n devient suffisamment grande.

Conclusion : La performance asymptotique (le comportement quand $n \to \infty$) est plus déterminante que les constantes multiplicatives liées au matériel.

3 Resume Lecture 2

4 Lecture 2 : Tri et Analyse de Complexité

4.1 Le Problème du Tri

Entrée : Une séquence de n nombres $\langle a_1, a_2, \ldots, a_n \rangle$.
Sortie : Une permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ de la séquence d'entrée telle que $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

4.2 Tri par Insertion (Insertion Sort)

Le tri par insertion fonctionne comme le tri d'une main de cartes à jouer. On parcourt les éléments un par un et on insère chaque nouvel élément à sa place correcte parmi ceux déjà triés (à gauche).

Algorithm 1 Insertion-Sort(A)

```
for j = 2 to A.length do
    key = A[j]
    i = j − 1
    while i > 0 and A[i] > key do
        A[i + 1] = A[i]
        i = i − 1
    end while
    A[i + 1] = key
end for
```

4.3 Preuve de Correction : Invariant de Boucle

Pour prouver qu'un algorithme itératif est correct, on utilise un **invariant de boucle**. C'est une propriété qui doit être vraie à trois moments clés :

1. **Initialisation :** Vraie avant la première itération.
2. **Conservation (Maintenance) :** Si elle est vraie avant une itération, elle reste vraie avant la suivante.
3. **Terminaison :** À la fin de la boucle, l'invariant fournit la propriété utile pour prouver la correction.

Application au Tri par Insertion : *Invariant :* Au début de chaque itération de la boucle 'for' indexée par j, le sous-tableau $A[1 \dots j-1]$ contient les éléments qui étaient initialement dans $A[1 \dots j-1]$, mais triés.

4.4 Analyse de Complexité (Modèle RAM)

Nous utilisons le modèle **Random-Access Machine (RAM)** où chaque instruction simple (arithmétique, accès mémoire, test) prend un temps constant.

- **Meilleur cas (Best case) :** Le tableau est déjà trié. La boucle 'while' ne s'exécute jamais. Complexité : $\Theta(n)$.
- **Pire cas (Worst case) :** Le tableau est trié en ordre inverse. La boucle 'while' s'exécute $j-1$ fois pour chaque j.

$$T(n) = \sum_{j=2}^{n} \Theta(j) = \Theta(n^2)$$

Nous nous concentrons généralement sur le **pire cas** car il fournit une garantie supérieure sur le temps d'exécution.

5 Lecture 3 : Diviser pour Régner (Merge Sort)

5.1 Le paradigme Diviser pour Régner

Cette approche récursive pour résoudre un problème se décompose en trois étapes :

1. **Diviser** le problème en plusieurs sous-problèmes qui sont des instances plus petites du même problème.
2. **Régner** sur les sous-problèmes en les résolvant récursivement. (Cas de base : si le problème est assez petit, on le résout directement).
3. **Combiner** les solutions des sous-problèmes pour obtenir la solution du problème original.

5.2 Tri Fusion (Merge Sort)

L'algorithme applique ce paradigme au tri :

- **Diviser :** Séparer le tableau de n éléments en deux sous-tableaux de taille $n/2$.
- **Régner :** Trier récursivement les deux sous-tableaux avec MERGE-SORT.
- **Combiner :** Fusionner (MERGE) les deux sous-tableaux triés pour produire un unique tableau trié.

5.2.1 La procédure Merge (Fusion)

L'étape clé est la fusion de deux séquences triées.

- On compare les têtes des deux listes et on place le plus petit élément dans le tableau de sortie.
- Cette opération prend un temps linéaire $\Theta(n)$ où n est le nombre total d'éléments à fusionner.

5.3 Analyse du Tri Fusion

Soit $T(n)$ le temps pour trier n éléments.

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases} \tag{1}$$

- $2T(n/2)$ correspond à la résolution des deux sous-problèmes.
- $\Theta(n)$ correspond au temps de la division et surtout de la fusion (Combiner).

En résolvant cette récurrence (par exemple via l'arbre de récursion ou le *Master Theorem* vu plus tard), on obtient :

$$T(n) = \Theta(n \log n)$$

Conclusion : Pour de grandes valeurs de n, le Tri Fusion ($\Theta(n \log n)$) est nettement plus rapide que le Tri par Insertion ($\Theta(n^2)$).

6 resume lecture 4 : Analyse et Maximum Subarray

6.1 Divide-and-Conquer Analysis

The running time $T(n)$ of divide-and-conquer algorithms is typically described by a recurrence:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

Where $D(n)$ is the division time and $C(n)$ the combination time. For **Merge Sort**, this yields $T(n) = 2T(n/2) + \Theta(n)$, resulting in $T(n) = \Theta(n \log n)$.

6.1.1 Solving Recurrences

There are three main methods to solve these equations:

6.1.2 Substitution Method

We guess the form of the solution and use mathematical induction to find the constants and prove the solution.

Example: $T(n) = T(n/4 + 1) + T(3n/4 - 1) + \Theta(1)$ *Problem:* Prove $T(n) = O(n)$.

Proof. We want to show $T(n) \leq cn - d$ for some constants $c, d > 0$. Assume it holds for all values smaller than n.

$$\begin{aligned} T(n) &\leq c(n/4 + 1) - d + c(3n/4 - 1) - d + k \quad (\text{where } \Theta(1) \leq k) \\ &= cn/4 + c - d + 3cn/4 - c - d + k \\ &= cn - 2d + k \end{aligned}$$

To satisfy $T(n) \leq cn - d$, we need $cn - 2d + k \leq cn - d$, which simplifies to $d \geq k$. Thus, $T(n) = O(n)$ for a sufficiently large constant d. □

6.1.3 Recursion Trees

Visualizes the cost at each level of subproblems. It is useful for generating a "guess" that can later be verified by the substitution method.

6.1.4 Master Method

A "cookbook" for recurrences of the form $T(n) = aT(n/b) + f(n)$. We compare $f(n)$ with $n^{\log_b a}$:

1. If $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$, then $T(n) = \Theta(f(n))$.

6.2 Maximum Subarray Problem

The goal is to find a contiguous subarray with the maximum sum within an array A.

6.2.1 Divide-and-Conquer Approach

We split the array into $A[low..mid]$ and $A[mid+1..high]$. The maximum subarray is either:

1. Entirely in the left half.
2. Entirely in the right half.
3. **Crossing the midpoint**.

The function `Find-Max-Crossing-Subarray` scans from mid to the left and from $mid+1$ to the right in $\Theta(n)$ time. The recurrence is $T(n) = 2T(n/2) + \Theta(n)$, giving:

$$\boxed{T(n) = \Theta(n \log n)}$$

7 resume lecture 5 : Algorithmes Divide-and-Conquer & Heaps

7.1 Matrix Multiplication

Given two $n \times n$ matrices A and B, the product $C = A \cdot B$ has $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$.

7.1.1 Naive vs. Strassen's Algorithm

- **Naive Algorithm:** Three nested loops, resulting in $\Theta(n^3)$.
- **Simple Divide-and-Conquer:** Splitting matrices into 4 sub-matrices of size $n/2$. $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$. No improvement.
- **Strassen's Algorithm:** Uses algebraic tricks to compute only **7 multiplications** instead of 8.

  $$T(n) = 7T(n/2) + \Theta(n^2)$$

  Using Master Theorem Case 1 ($\log_2 7 \approx 2.81$):

  $$\boxed{T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})}$$

7.2 Heaps and Heapsort

Heapsort combines the benefits of Merge Sort ($O(n \log n)$) and Insertion Sort (in-place).

7.2.1 Binary Heap Structure

A heap is an almost complete binary tree stored in an array.

- **Root:** $A[1]$
- **Max-Heap Property:** $A[\text{Parent}(i)] \geq A[i]$. The largest element is at the root.
- **Height:** A heap of n elements has height $\Theta(\log n)$.

7.2.2 Key Operations

7.2.3 Max-Heapify

Maintains the heap property. If an element is smaller than its children, it "sinks" down the tree.

- **Complexity:** $O(\log n)$.

7.2.4 Build-Max-Heap

Converts an unordered array into a Max-Heap by calling *Max-Heapify* on all non-leaf nodes.

- **Complexity:** Although it makes $O(n)$ calls to a $\log n$ function, the tighter analysis shows it is **linear**.

$$\boxed{T(n) = \Theta(n)}$$

```
function BUILD-MAX-HEAP(A)
    A.heap_size ← A.length
    for i ← ⌊A.length/2⌋ downto 1 do
        MAX-HEAPIFY(A, i)
    end for
end function
```

8 Lecture 6: Matrix Multiplication and Heaps

Note: The first part of this lecture is a recall on the Maximum-Subarray Problem. As requested, this summary focuses on the new concepts: Matrix Multiplication and Heaps.

8.1 Matrix Multiplication

Problem Definition: Given two $n \times n$ (square) matrices A and B, compute their product $C = A \cdot B$.

- **Naive Algorithm:** Computes the product using three nested loops, calculating $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$. Running time is $\Theta(n^3)$ and space is $\Theta(n^2)$.
- **Simple Divide-and-Conquer:** Divides matrices into $n/2 \times n/2$ submatrices. It performs 8 recursive multiplications. The recurrence is $T(n) = 8T(n/2) + \Theta(n^2)$, which still yields $T(n) = \Theta(n^3)$.
- **Strassen's Algorithm:** A smarter divide-and-conquer approach that reduces the number of recursive multiplications from 8 to 7, at the cost of some additional matrix additions.
  - Recurrence: $T(n) = 7T(n/2) + \Theta(n^2)$.
  - Running Time: Using the Master Theorem, $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.8})$, which asymptotically beats the naive $\Theta(n^3)$ approach.

8.2 Heaps and Heapsort

Heapsort is an in-place sorting algorithm that runs in $\mathcal{O}(n \log n)$ worst-case time, combining the best properties of Insertion Sort (in-place) and Merge Sort ($\mathcal{O}(n \log n)$).

8.2.1 The Heap Data Structure

A binary heap is a nearly complete binary tree.

- **Array representation:** It can be stored in an array without pointers. For a node at index i:
  - $\text{LEFT}(i) = 2i$
  - $\text{RIGHT}(i) = 2i + 1$
  - $\text{PARENT}(i) = \lfloor i/2 \rfloor$

- **Max-Heap Property:** For every node i other than the root, the key of its parent is greater than or equal to its own key ($A[\text{PARENT}(i)] \geq A[i]$). The maximum element is always at the root.
- **Height:** The height of a heap of size n is $\Theta(\log n)$.

8.2.2 Heap Operations

- MAX-HEAPIFY(A, I): Ensures the max-heap property for the subtree rooted at i, assuming its left and right subtrees are already max-heaps. It propagates the element down the tree and takes $\Theta(\text{height of } i) = \mathcal{O}(\log n)$ time.
- BUILD-MAX-HEAP(A): Constructs a max-heap from an unordered array of length n. By a tight analysis (since time is linear in the height of the node), it runs in $\mathcal{O}(n)$ time rather than $\mathcal{O}(n \log n)$.

8.2.3 The Heapsort Algorithm

1. Build a max-heap from the input array.
2. Starting with the root (the maximum element), place it into its correct position by swapping it with the last element in the array.
3. "Discard" this last node by decreasing the heap size, and call MAX-HEAPIFY on the new root to restore the max-heap property.
4. **Total Running Time:** $\mathcal{O}(n \log n)$ because building the heap takes $\mathcal{O}(n)$ and we call MAX-HEAPIFY ($\mathcal{O}(\log n)$) $n - 1$ times.

8.3 Priority Queues

A priority queue maintains a dynamic set of elements where each element has an associated "key". Heaps efficiently implement priority queues. A max-priority queue supports the following operations:

- MAXIMUM(S): Returns the element with the largest key in $\mathcal{O}(1)$ time.
- EXTRACT-MAX(S): Removes and returns the maximum element in $\mathcal{O}(\log n)$ time.
- INCREASE-KEY(S, X, K): Increases the key of element x to k and traverses upward to restore the heap property in $\mathcal{O}(\log n)$ time.
- INSERT(S, X): Inserts element x into the set in $\mathcal{O}(\log n)$ time.

9 Resume lecture 7: Elementary Data Structures

Note: The beginning of this lecture is devoted to important recalls on Heaps, Heapsort, and Priority Queues. Since these concepts were already covered in Lecture 6, this summary focuses exclusively on the newly introduced elementary data structures.

9.1 Dynamic Sets

Data structures are used to implement dynamic sets of elements. The operations on these sets are generally grouped into two categories:

- **Modifying operations:** Insert, delete, etc.
- **Query operations:** Search, minimum, maximum, etc.

9.2 Stacks

A stack is a dynamic data structure that operates on the **LIFO** (Last-In, First-Out) principle.

- **Operations:**
  - PUSH(S, x): Inserts element x at the top of the stack.
  - POP(S): Removes and returns the element at the top of the stack.
  - STACK-EMPTY(S): Checks whether the stack is empty.
- **Implementation:** Can be implemented using an array `S` with an attribute `S.top` that points to the most recently inserted element. The bottom of the stack is at `S[1]`.
- **Complexity:** All of these operations execute in constant time $\mathcal{O}(1)$.

9.3 Queues

A queue operates on the **FIFO** (First-In, First-Out) principle, exactly like a waiting line in real life.

- **Operations:**
  - ENQUEUE(Q, x): Adds element x to the tail (end) of the queue.
  - DEQUEUE(Q): Removes and returns the element at the head (front) of the queue.
- **Implementation:** Can be implemented using an array `Q` with two pointers:
  - `Q.head`: Points to the first element in the queue (the next one to be dequeued).
  - `Q.tail`: Points to the next available open slot where a newly enqueued element will be placed.
- **Complexity:** Both ENQUEUE and DEQUEUE operations execute in constant time $\mathcal{O}(1)$.

9.4 Linked Lists

Unlike arrays, where the linear order is determined by array indices, the linear order in a linked list is determined by **pointers** stored within each object. They can be singly linked (pointers to the next element only) or doubly linked (pointers to both the next and previous elements).

- **Operations and Complexities:**
  - **Search (LIST-SEARCH(L, k))**: Traverses the list sequentially to find the key k. Running time is $\mathcal{O}(n)$.
  - **Insertion (LIST-INSERT(L, x))**: Splices a new element into the list. Running time is $\mathcal{O}(1)$.
  - **Deletion (LIST-DELETE(L, x))**: Removes an element from the list. Running time is $\mathcal{O}(1)$ (for a doubly linked list, assuming we already have the pointer to the object we want to delete).
- **Sentinels:** A sentinel is a dummy object (e.g., `L.nil`) used to represent the beginning or the end of a list. They greatly simplify the code by eliminating the need to handle boundary conditions (such as checking for `NULL` pointers at the head or tail).

Algorithm 2 reverse of linked list

```
procedure REVERSE-LIST(L)
    prev ← NIL
    curr ← L.head
    while curr ≠ NIL do
        next_node ← curr.next
        curr.next ← prev
        prev ← curr
        curr ← next_node
    end while
    L.head ← prev
end procedure
```

9.4.1 An exemple on LinkedList of how to reverse it

Here is an exemple on how to reverse the linkedList in time complexity $\Theta(n)$ and space complexity $\Theta(1)$:

9.5 Conclusion on Elementary Data Structures

Stacks, Queues, and Linked Lists are excellent for specific operations (like insertions and deletions which run in $\mathcal{O}(1)$ time), but they are inherently **bad at search** operations, as finding a specific element generally requires $\mathcal{O}(n)$ time.